



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

02.1: Software-Defined Networking (SDN)

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Foundational Technology Enablers

Objective: To introduce the foundational concepts and technologies that enable the flexible, agile, and intelligent network infrastructure required for 6G and beyond.

Lesson 2.1: Software-Defined Networking (SDN)

- **Duration:** 4 Hours
- **Focus:** Understanding the core principles of SDN, including the decoupling of control and data planes, the role of the centralized controller, and how it enables programmatic network management and agility.

Lesson 2.2: Network Functions Virtualization (NFV)

- **Duration:** 4 Hours
- **Focus:** Exploring how network functions are virtualized and run as software on commodity hardware, leading to cost reduction, increased flexibility, and faster service deployment. We'll also examine the synergy between SDN and NFV.

Lesson 2.3: The Cloud-to-Edge Continuum

- **Duration:** 2 Hours
- **Focus:** Examining the distributed computing paradigm that underpins 6G, including the roles of centralized cloud computing and highly distributed edge computing, and how they seamlessly integrate to form a powerful, intelligent infrastructure fabric.

Software-Defined Networking



The Case for a New Architecture

Recap from Module 1: We saw that 5G introduced a Service-Based Architecture (SBA) and Control/User Plane Separation (CUPS). These are key steps towards a more flexible, programmable network.

Today's Goal: To understand the fundamental paradigm that underpins this shift:
Software-Defined Networking (SDN).

We will start by answering a critical question: What was so wrong with traditional networks that we needed a complete architectural revolution?

Traditional Network Architecture: Distributed Control

The Paradigm: In a traditional IP network, every network device (router, switch) is a vertically integrated, closed system.

Each device contains:

- **Data Plane:** The hardware responsible for forwarding packets at high speed (e.g., ASICs).
- **Control Plane:** The software "brain" that runs complex, distributed routing protocols (e.g., OSPF, BGP) to build its own map of the network.
- **Management Plane:** The interface for human operators to configure the device (e.g., CLI).

Key Characteristic: The control logic is **distributed**. Each device makes independent decisions based on the information it exchanges with its immediate neighbors.

1. Data Plane (Forwarding Plane)

The "Forwarding Hardware": This is where the actual work of moving data occurs.

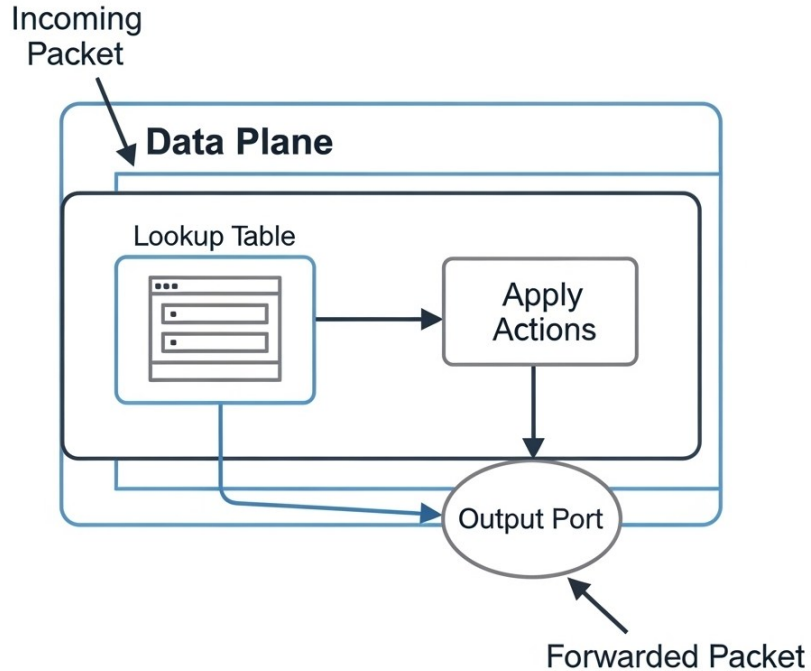
Function: Purely focused on **packet processing** at **line speed**. Its primary tasks are:

- **Look up destination:** Consults a pre-built forwarding table (e.g., a FIB - Forwarding Information Base) to determine the next hop for an incoming packet.
- **Apply actions:** Executes specific actions based on the lookup result (e.g., forward the packet, drop it, modify its headers, apply QoS policies).
- **Send to output port:** Directs the packet to the appropriate outgoing interface.

Measurement: Performance is typically measured in **packets-per-second (PPS)** or bits-per-second (BPS).

Key Characteristic: Optimized for speed and efficiency, often implemented in **specialized hardware (ASICs)**.

Data Plane (Forwarding Plane)



2. Control Plane

The "Distributed Algorithm": This is the "brain" of the network device, **responsible for making decisions about how traffic should flow.**

Function: Determines the **network's state** and builds the forwarding tables used by the data plane. It runs various protocols to achieve this:

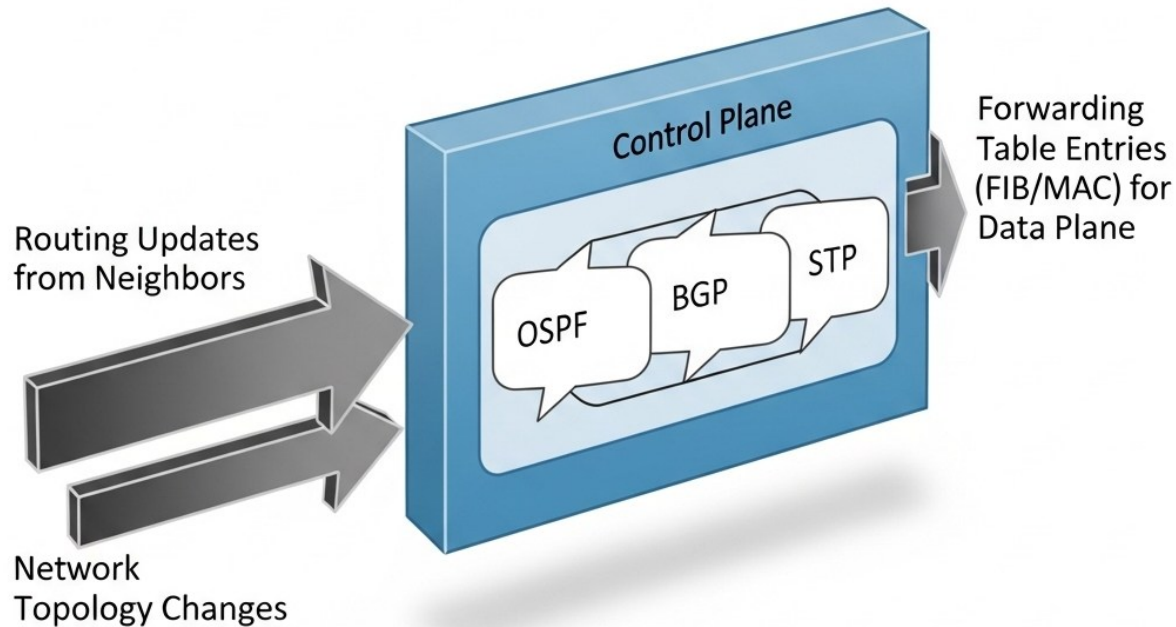
- **Routing Protocols:** OSPF (Open Shortest Path First), BGP (Border Gateway Protocol) to exchange routing information with other devices and calculate optimal paths.
- **Switching Protocols:** STP (Spanning Tree Protocol) to prevent loops in Layer 2 networks.
- **Other Protocols:** ARP, DHCP, etc.

Input: Routing updates from neighboring devices, network topology changes.

Output: Populates the **forwarding tables (FIB/MAC tables)** in the data plane.

Key Characteristic: Often implemented **in software running on a general-purpose CPU** within the device.

Control Plane



3. Management Plane

The "Configuration Interface": This plane provides the means for network operators to interact with and **manage the device**.

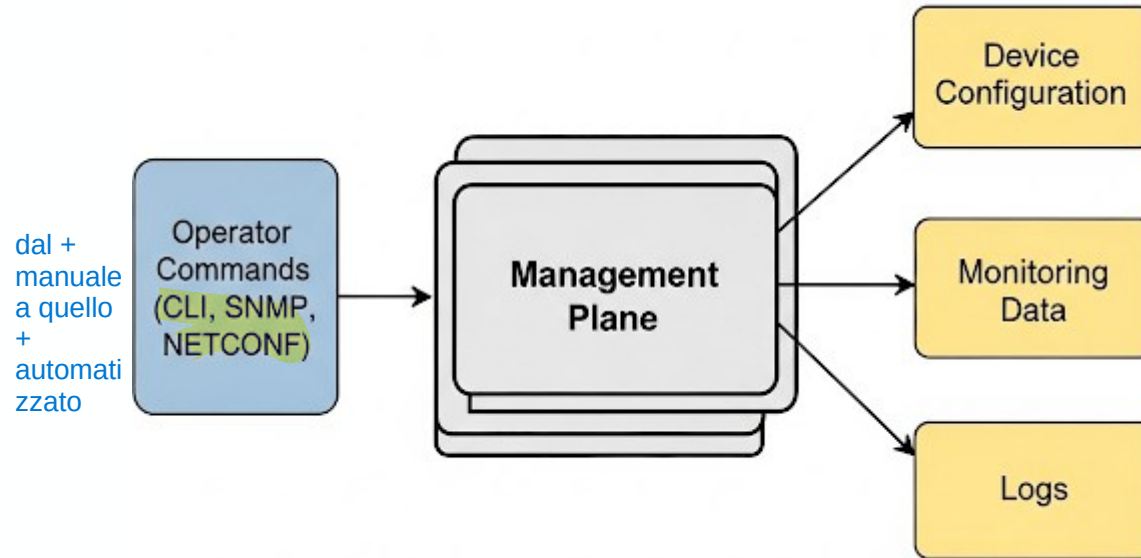
Function: Allows for **configuration, monitoring, and troubleshooting** of the network device.

Tools/Interfaces:

- **CLI (Command-Line Interface):** The most common text-based interface for direct interaction.
- **SNMP (Simple Network Management Protocol):** Used for monitoring device health and **performance**.
- **NETCONF/RESTCONF:** Modern, programmatic interfaces for automated configuration and telemetry.

Key Characteristic: Typically accessed remotely or locally by network administrators.

Management Plane



Limitation 1: Crippling Complexity

Box-by-Box Configuration: Every device must be configured individually using low-level, vendor-specific Command-Line Interfaces (CLIs). This is slow and error-prone.

Inconsistent Policies: With thousands of devices, ensuring consistent security and QoS policies across the network is a major operational challenge. A single misconfiguration on one router can violate an end-to-end policy.

Human-Scale Time: Configuration is a manual process. Complex network-wide changes can take weeks to plan and safely implement, hindering business agility.

Limitation 2: The Agility Gap (Static Policies)

The Problem: The network cannot keep pace with the speed of modern, dynamic applications.

Cloud & Virtualization: In a data center, a developer can provision a new virtual machine or application in **seconds** via an API call.

The Network Bottleneck: But provisioning the required network connectivity (VLANs, firewall rules, QoS policies) for that new application can **take days or weeks** because it requires a manual, static change management process.

The Mismatch: This agility gap makes the network an obstacle to innovation rather than an enabler.

Limitation 3: Vendor Lock-In

Vertically Integrated Systems: The control plane software on a device is proprietary and tightly coupled to the underlying hardware.

Stifled Innovation: It is impossible for a network owner to deploy a new routing algorithm or a novel security feature. You are limited to whatever features the vendor provides in their next software release.

High Costs: Lack of competition and the need for vendor-specific training and certification lead to high operational (OpEx) and capital (CapEx) expenditures. You cannot buy "white-box" hardware and run the control software of your choice.

The Core Idea: Decoupling Control and Data Planes



The Core Idea of SDN: Decoupling

Top Piece (Brain): Control Plane Logic (Routing, Policy). This piece is physically moved out of the device into a software controller.

Bottom Piece (Muscle): Data Plane (Packet Forwarding Hardware). This piece remains in the device, simplified and streamlined.

Software-Defined Networking logically separates the network's control plane from its data plane. This simple but powerful idea is the key to solving the limitations of traditional networks.

The Result: A Logically Centralized Architecture

The Control Plane is Centralized: The "brains" from all network devices are moved to a single software entity called the **SDN Controller**.

The Data Plane is Simplified: The network devices become simple, programmable forwarding elements. Their only job is to execute the instructions given to them by the controller.

A Standard Protocol Connects Them: The controller communicates its instructions to the switches using an open, standard protocol (the "Southbound API").

Three Benefits:

1. **Global Visibility and Centralized Control**
2. **Network Programmability & Automation**
3. **Fostering Innovation**

Benefit 1: Global Visibility and Centralized Control

The Controller as Oracle: The SDN controller builds and maintains a complete, real-time, global view of the entire network topology and traffic state.

Globally Optimal Decisions: With this global view, the controller can make decisions that are optimal for the entire network, not just for a single router's local view. This is impossible in traditional distributed routing.

Simplified Management: A network administrator interacts with a single controller to manage the entire network, drastically reducing complexity.

Benefit 2: Network Programmability & Automation

Network as Code or Networks as a Service: The controller exposes high-level APIs (Northbound APIs) to applications.

Automation: Network administrators **can write simple scripts** or applications to automate repetitive tasks that previously required manual CLI configuration on hundreds of devices.

Intent-Based Networking: Applications can declare their *intent* (e.g., "isolate this group of VMs" or "guarantee 100 Mbps for this video stream"), and **the controller automatically translates that intent into the necessary low-level forwarding rules.**

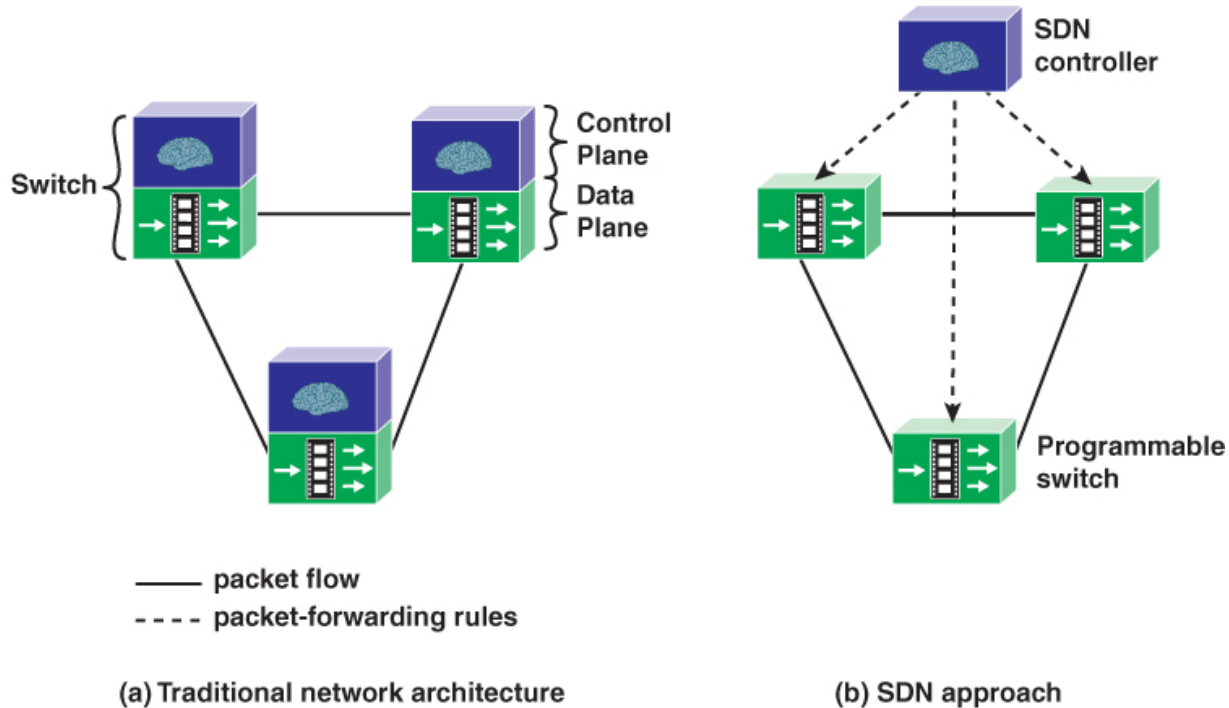
Benefit 3: Fostering Innovation

Open Interfaces: SDN promotes the use of open, standard interfaces (like OpenFlow) between the control and data planes.

Disaggregation: This allows network operators to break vendor lock-in. They can buy commodity "white-box" switches from one vendor and use an open-source or commercial controller from another.

Rapid Service Creation: New network functions and services can be deployed quickly as applications on top of the controller, without needing to wait for a vendor to add a feature to their proprietary router software.

Visualizing the Transformation



Traditional vs. SDN: A Paradigm Shift

Feature	Traditional Networking	Software-Defined Networking (SDN)
Control Plane	Distributed, on every device	Logically Centralized, in the Controller
Device Intelligence	High (runs complex protocols)	Low (simple forwarding element)
Configuration	Manual, box-by-box, vendor -specific	Programmatic, network-wide, open
Network View	Fragmented, local	Global, centralized
Innovation	Slow, vendor-driven	Fast, software-driven

SDN Architecture



The Three Layers of the SDN Architecture

Core Concept: Separation between the control plane and the data plane.

Separation Approach:

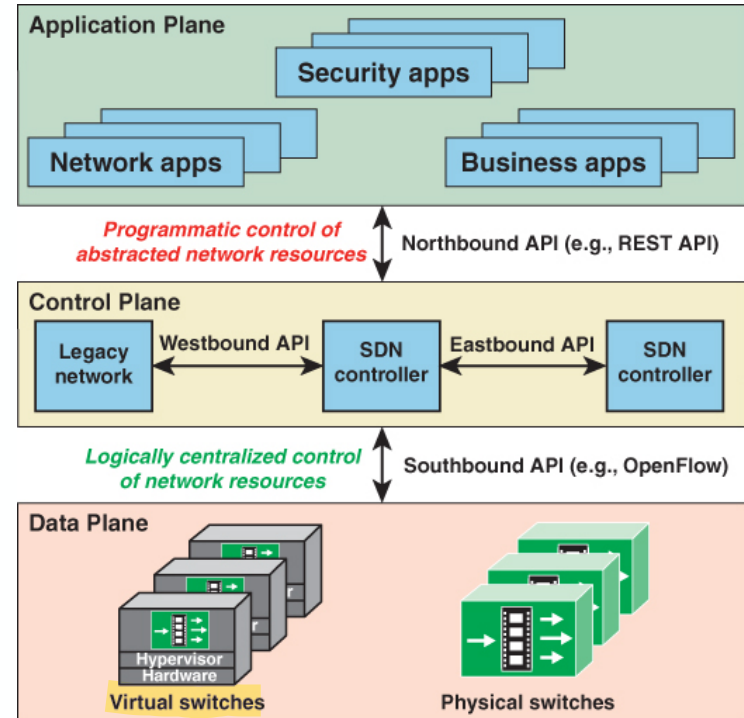
- Similar to the OSI stack.
- Facilitates development and testing of new protocols and hardware without changing the entire stack.
- Each layer provides an interface to its neighbors.

Three Layers:

- Application Plane
- Control Plane
- Data Plane

Four Interfaces

- Northbound & Southbound
- Eastbound & Westbound



Infrastructure Layer (Data Plane)

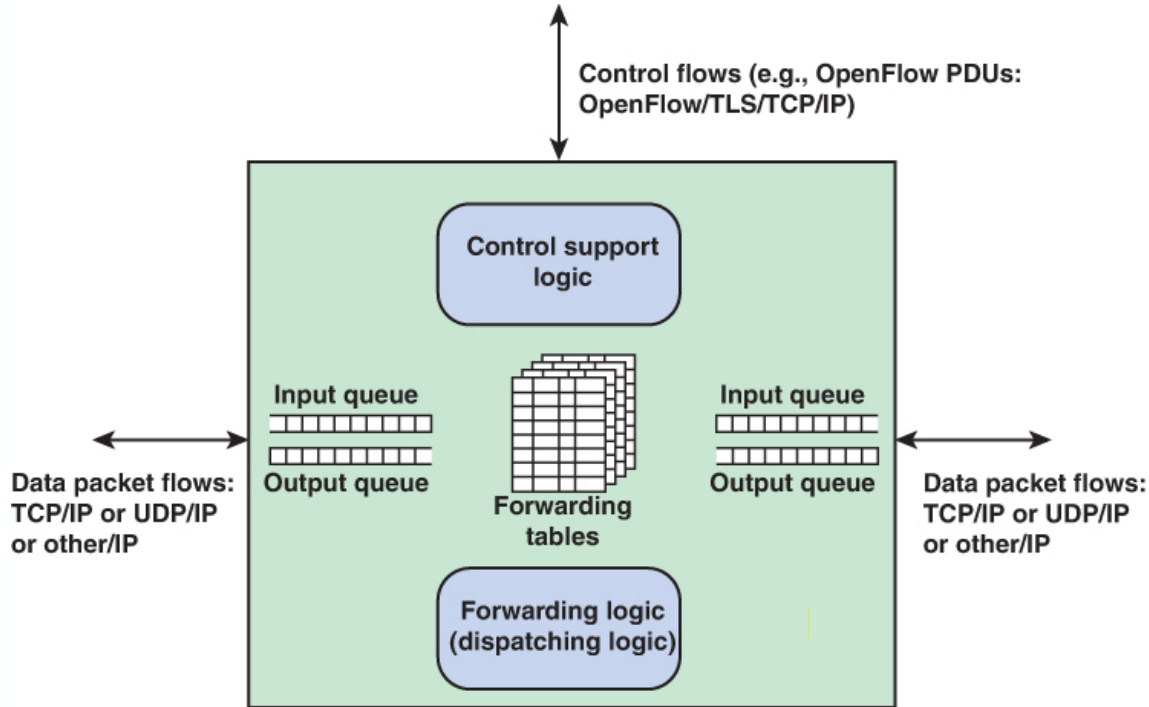
Components: This layer comprises the physical network devices (e.g., switches, routers, access points) and virtual network devices (e.g., virtual switches in hypervisors).

Role:

- Their primary role is to act as **simple, programmable forwarding elements**.
- They execute the forwarding instructions (flow rules) pushed down by the Control Layer.
- They also **expose their forwarding capabilities** and **report their state** (e.g., port status, traffic statistics) back to the Control Layer.

Key Characteristic: **Focus on high-speed packet processing**, often implemented in specialized hardware (ASICs) or optimized software.

Infrastructure Layer (Data Plane)



Control Layer (SDN Controller / Network OS)

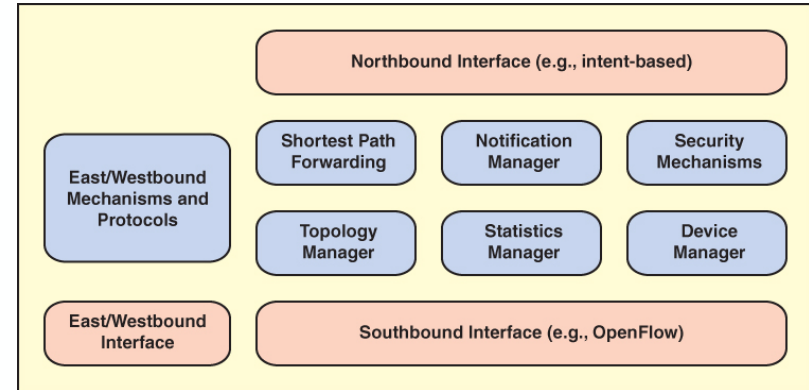
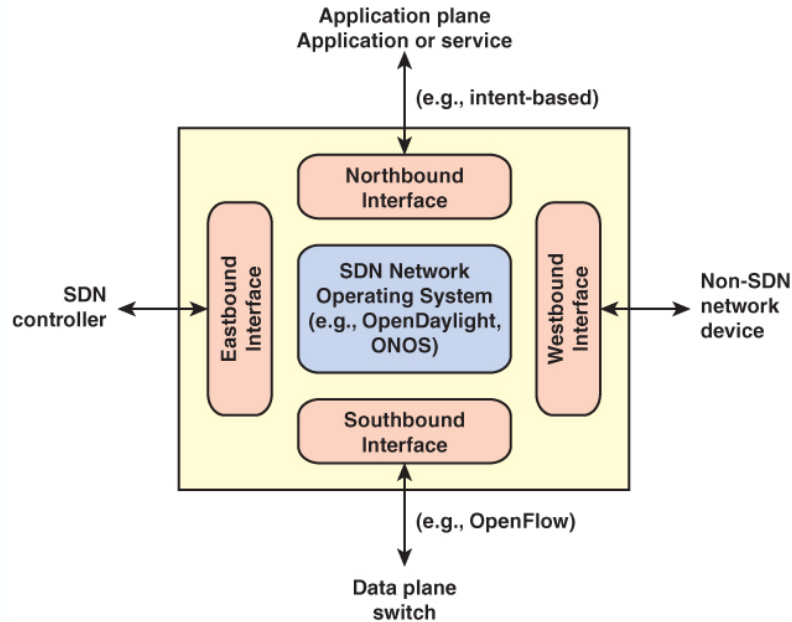
Components: This is the "brain" of the SDN architecture, typically implemented as a software platform running on commodity servers. It's often referred to as the **Network Operating System (NOS)**.

Role:

- **Maintain a consistent, abstract view of the entire network topology and state.** It collects information from all devices in the Infrastructure Layer.
- **Translate high-level application requirements** (from the Application Layer) into low-level, device-specific forwarding instructions for the Data Plane.
- **Manage network resources, handle device discovery,** and enforce policies.

Key Characteristic: Logically centralized, provides northbound (to applications) and southbound (to infrastructure) APIs.

Control Layer (SDN Controller / Network OS)



Application Layer

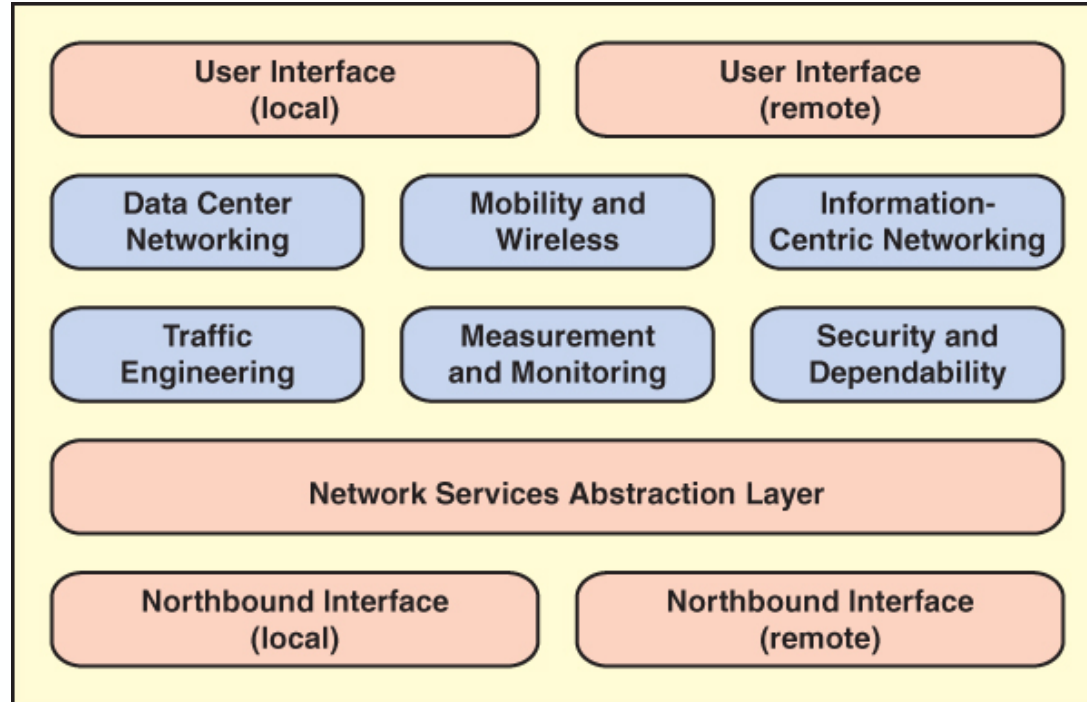
Components: This layer consists of various network services and applications that run on top of the SDN Controller. These applications implement the actual business logic and network policies.

Role:

- They leverage the abstract view and programmable interface provided by the Control Layer to **define and implement network behavior**.
- Examples include:
 - **Routing Algorithms:** Implementing custom routing policies (e.g., traffic engineering, shortest path).
 - **Security Policies:** Implementing firewalls, intrusion detection, and access control lists.
 - **Load Balancers:** Distributing traffic efficiently across servers.
 - **Network Monitoring & Analytics:** Collecting and analyzing network performance data.
 - **Cloud Orchestration Systems:** Integrating network provisioning with virtual machine deployment.

Key Characteristic: High-level, abstract, and often vendor-agnostic, allowing for rapid innovation.

SDN Application Plane Functions



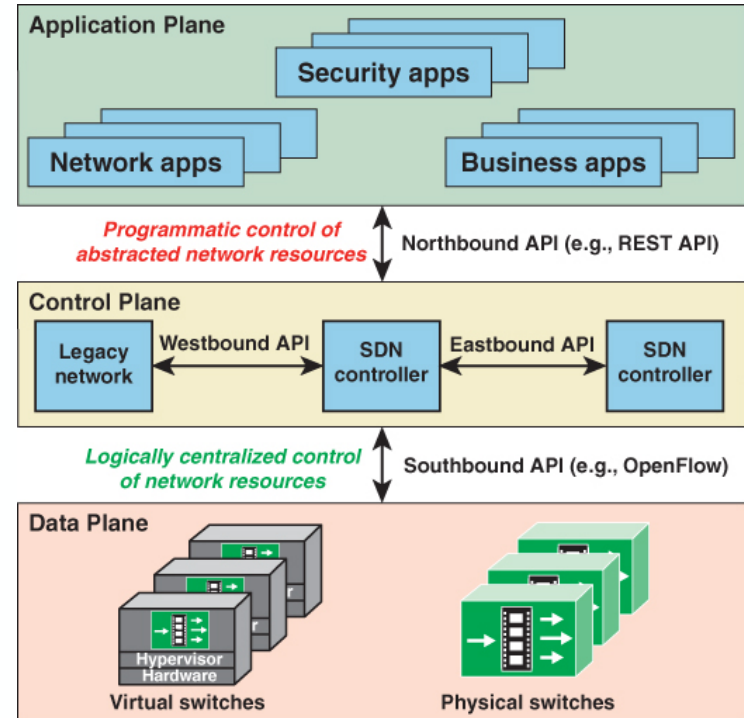
Northbound and Southbound Interfaces

Northbound Interface:

- Applications convey network requirements and desired behavior to the SDN controller via a northbound API.
- Currently no standard.
 - Example: REpresentational State Transfer (REST) API for applications to communicate their requirements to the network.

Southbound Interface:

- Used by the controller to program the data plane.
- Standardized by OpenFlow or NETCONF protocols.



The Southbound API

The Southbound API is the crucial interface that allows the SDN Controller to communicate with and program the underlying network devices (the Data Plane). While other protocols exist, **OpenFlow** has been the most influential and **widely adopted standard for this purpose**.

The OpenFlow Protocol

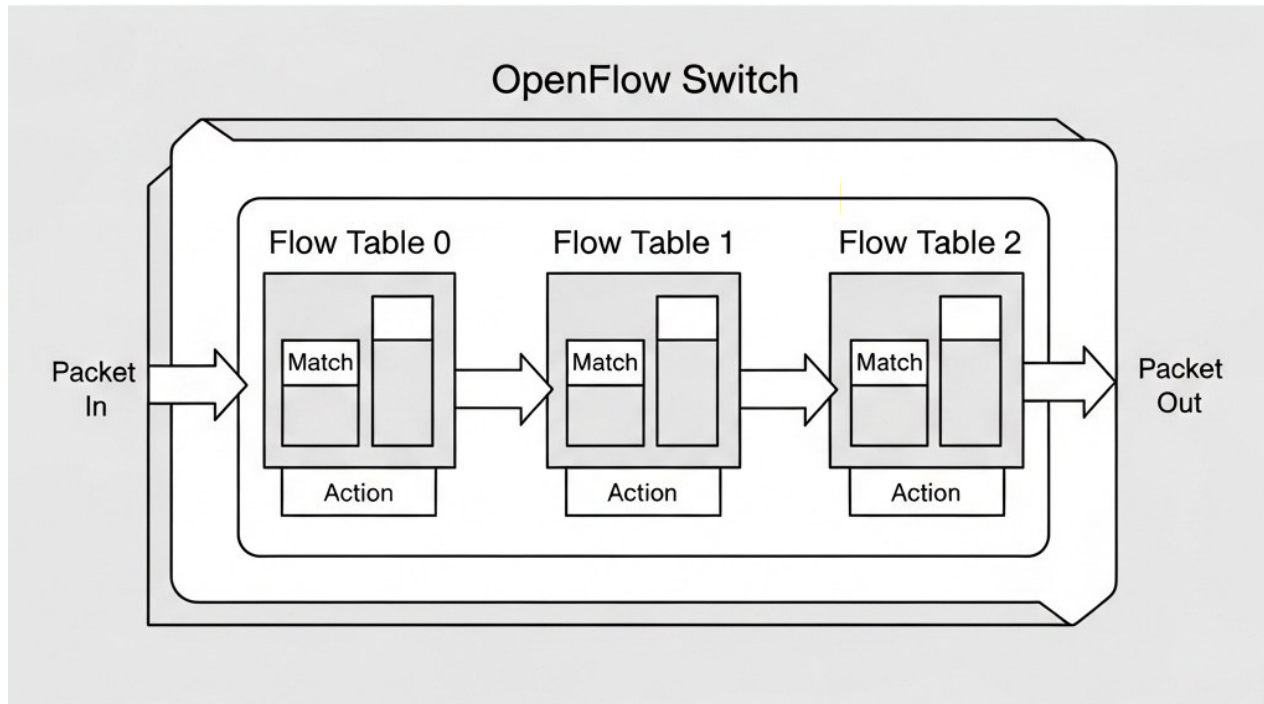
OpenFlow is a **standardized protocol** that enables the SDN Controller to securely communicate with, control, and manage **OpenFlow-enabled switches** (or "OpenFlow-capable devices").

It provides a granular way for the controller to define how packets are processed and forwarded by the data plane.

The OpenFlow Switch Model

- An OpenFlow switch is conceptually modeled as one or more **Flow Tables**.
- Each **Flow Table** contains a set of **Flow Entries (rules)**.
- A **Flow Entry** typically consists of three parts:
 1. **Match Fields:** Criteria to match incoming packets (e.g., **source IP, destination IP, port numbers, VLAN ID, ingress port**).
 2. **Instructions:** **Actions to perform if a packet matches the rule** (e.g., forward to a specific port, drop, modify header fields, send to the controller, go to the next table).
 3. **Counters:** Statistics about the packets matched by this entry.

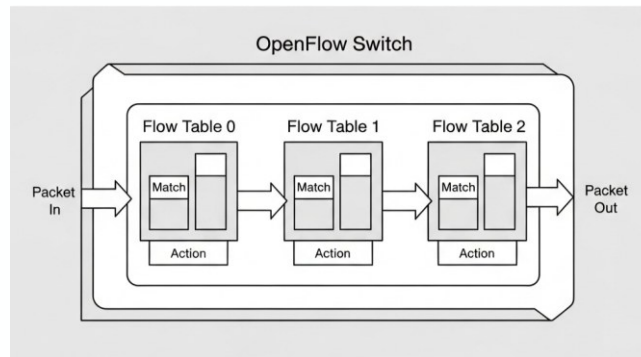
OpenFlow Switch



The Southbound API

The Matching Pipeline

- When a packet arrives at an OpenFlow switch, it is matched against the **Flow Entries** in the **Flow Tables** in a **pipeline sequence**.
- A rule can specify an action to forward the packet to the next table in the pipeline for further processing, allowing for complex, multi-stage packet handling.
- If no match is found in any table, the default behavior is often to send the packet to the controller (Packet-In message) for a decision.



The Southbound API

Key OpenFlow Message Types

OpenFlow defines various message types for communication between the Controller and the Switch

Controller-to-Switch Messages (Controller instructs Switch)

- **Flow-Mod:** The most common message. Used by the controller to **add, delete, or modify Flow Entries** in the switch's flow tables. This is how the network's forwarding behavior is programmed.
- **Packet-Out:** Instructs the switch to **send a specific packet** (either one received from the controller or one that was buffered by the switch) out of a particular port.

Switch-to-Controller Messages (Switch reports to Controller)

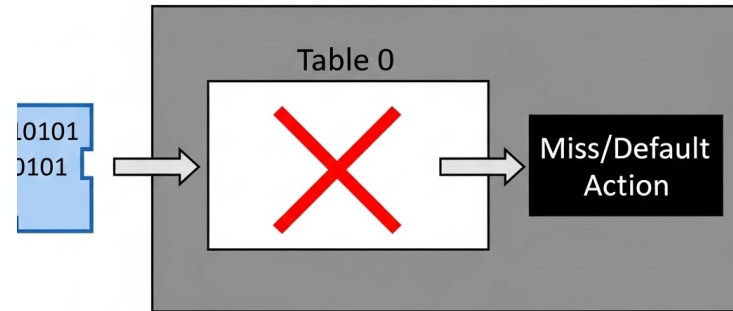
- **Packet-In:** Sent by the switch to the controller when a packet arrives that **does not match any existing Flow Entry** in its tables, or when a rule explicitly instructs the switch to send the packet to the controller. This allows the controller to make a decision for the new flow.
- **Port-Status:** Reports **changes in the status of a switch port** (e.g., port goes up/down, link failure). This helps the controller maintain an accurate view of the network topology.
- **Flow-Removed:** Informs the controller when a flow entry has been removed from a table (e.g., due to timeout).

The "First Packet" Journey: (1) Packet Arrival & Table Miss

When a new communication flow begins, the very first packet encounters a network that needs to learn how to handle it.

Packet Arrival & Table Miss

1. **Packet Arrives:** A packet, belonging to a new communication flow (e.g., the first packet of a new TCP connection), arrives at an **ingress port** of an OpenFlow-enabled switch.
2. **Pipeline Lookup Begins:** The switch immediately begins its **pipeline lookup process**, starting with the first flow table (typically Table 0).
3. **No Rule Matches:** The packet's header fields are compared against all the **Flow Entries** in the table. In this scenario, since it's the "first packet" of a new flow, there is **no existing rule that matches its characteristics**.

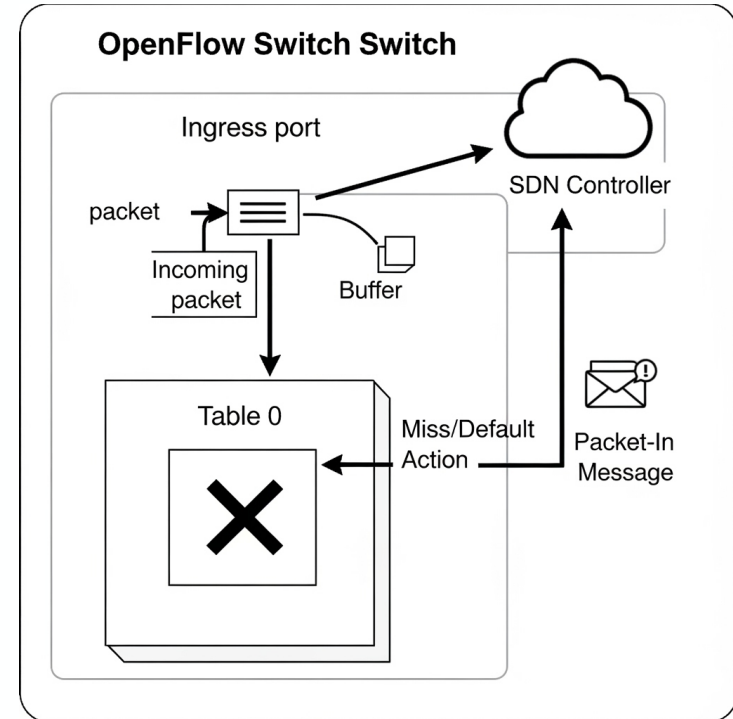


The "First Packet" Journey: (2) The Packet-In Event

When a switch encounters a packet, it doesn't know how to handle, it seeks guidance from the central intelligence – the SDN Controller.

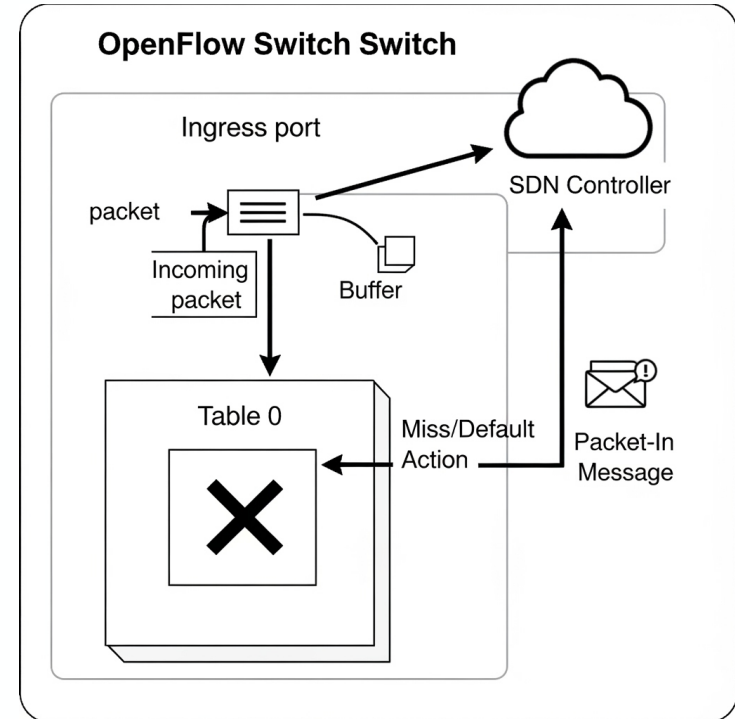
The Packet-In Event

- **Default Action Triggered:** Because of the **table miss** (no matching rule found), the switch's default action for unmatched packets is triggered. This action is typically configured to send the packet to the controller.
- **Packet-In Message:** The switch encapsulates the incoming packet's header (and sometimes a portion of the payload) into an **OpenFlow Packet-In message**.



The "First Packet" Journey: (2) The Packet-In Event

- **Message Sent to Controller:** This Packet-In message is then securely sent over the Southbound API (e.g., TLS over TCP) to the **SDN Controller**.
- **Packet Buffering:** To avoid dropping the initial packet(s) while waiting for the controller's decision, the switch typically **buffers the packet(s)** temporarily.

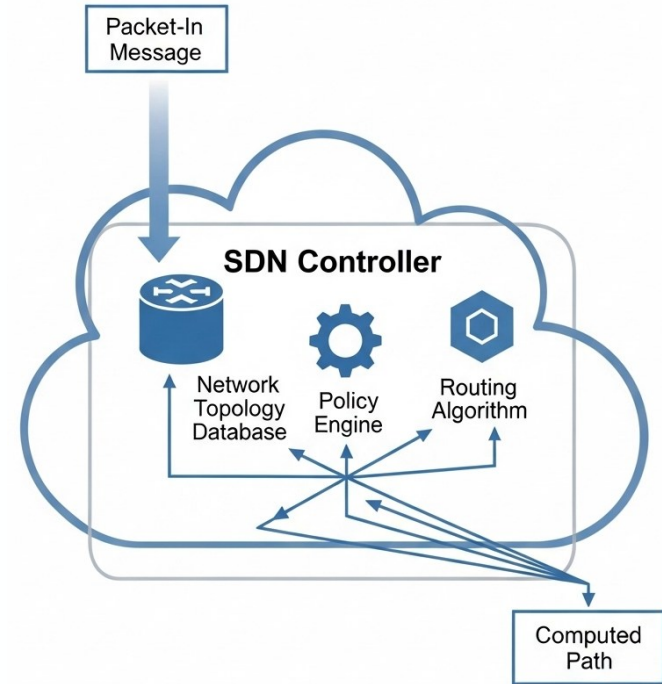


The "First Packet" Journey: (3) Controller Deliberation

Upon receiving a Packet-In message, the SDN Controller, with its global view, takes charge of deciding how the new traffic flow should be handled.

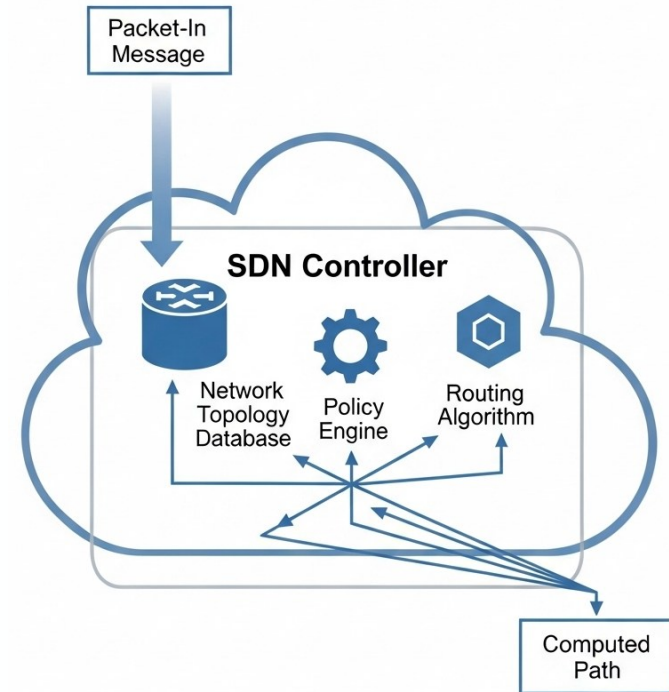
Controller Deliberation

1. **Event Reception:** The SDN Controller receives the **Packet-In message** from the switch.
2. **Application Notification:** The controller's internal mechanisms notify the relevant **network application** (e.g., a routing application, a security policy engine) about the new packet requiring a decision.



The "First Packet" Journey: (3) Controller Deliberation

3. **Analysis & Computation:** The application analyzes the packet header information (source/destination IPs, ports, etc.) provided in the Packet-In message.
4. **Global View & Policy Enforcement:** Using its **global view of the network topology** (collected from all connected switches) and pre-defined **security policies, QoS requirements, or routing algorithms**, the application computes the optimal forwarding path for this specific flow across the entire network.

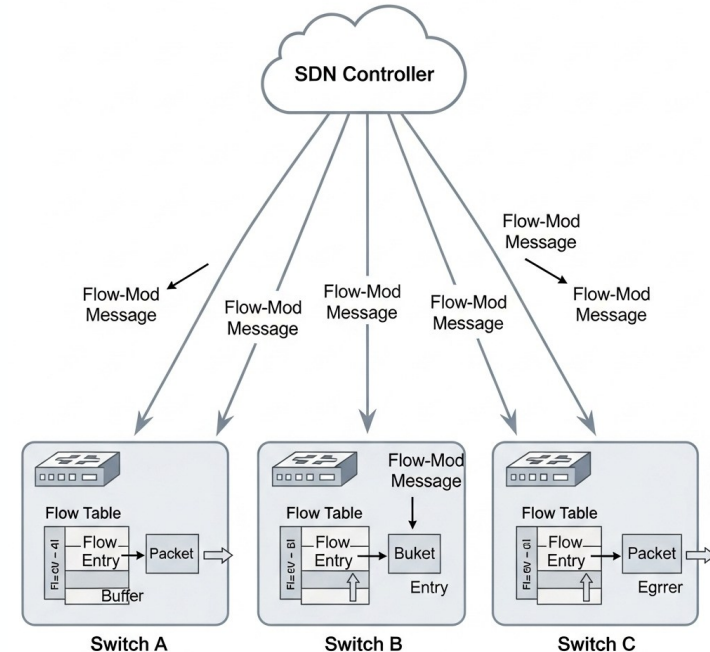


The 'First Packet' Journey: (4) Flow-Mod Installation

Once the controller has determined the path, it actively programs the network to ensure subsequent packets of the same flow are handled efficiently.

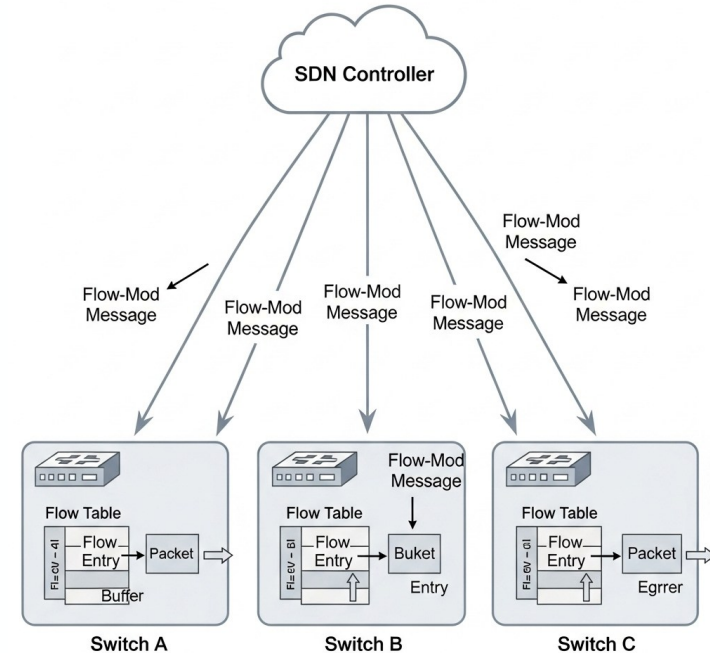
Flow-Mod Installation

- **Rule Generation:** Based on the computed forwarding path and associated policies, the SDN Controller generates specific **Flow Entries** (rules) for each relevant switch along the path.
- **Flow-Mod Messages:** The controller sends **Flow-Mod messages** to all the necessary OpenFlow-enabled switches along the newly computed path.



The 'First Packet' Journey: (4) Flow-Mod Installation

- **Rule Installation:** Each switch receives its respective Flow-Mod message and **installs the new Flow Entry** into its flow table(s). These rules instruct the switch on how to handle future packets belonging to this specific flow (e.g., match source IP X, destination IP Y, forward out Port Z).
- **Packet Release (Optional):** If the initial packet(s) were buffered by the ingress switch, the controller can also send a **Packet-Out message** to instruct that switch to release the buffered packet(s) onto the newly established path.

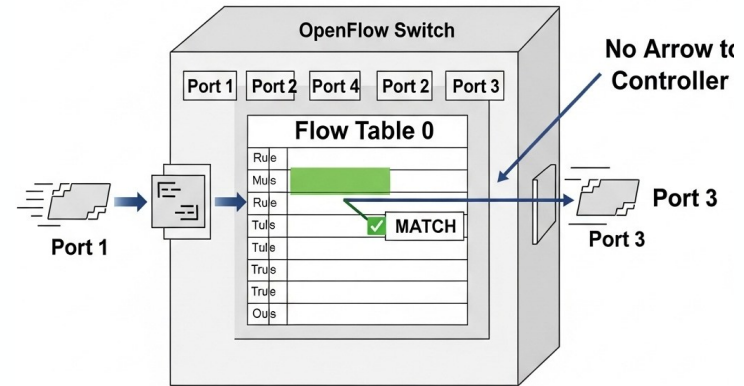


The 'First Packet' Journey: (5) Subsequent Packet Forwarding

After the initial setup, the network becomes highly efficient, handling ongoing traffic flows directly at the hardware level.

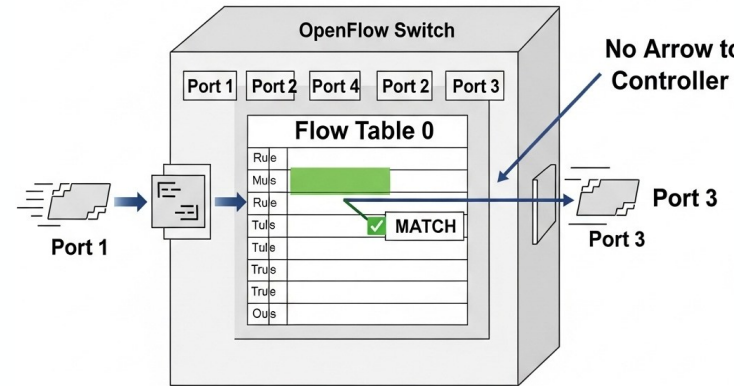
Subsequent Packet Forwarding

- **Packet Arrival (Same Flow):** The next packet (or any subsequent packet) of the **same communication flow** arrives at the ingress OpenFlow switch.
- **Immediate Match:** This time, when the packet enters the pipeline lookup, it finds an **immediate match** with the Flow Entry that was previously installed by the SDN Controller.



The 'First Packet' Journey: (5) Subsequent Packet Forwarding

- **Line-Rate Forwarding:** Because a match is found, the switch executes the associated instructions (e.g., forward out a specific port) directly in its data plane. **This happens at line-rate**, meaning at the maximum speed the hardware can support.
- **No Controller Interaction:** **Crucially, for all subsequent packets of this flow, there is no further interaction with the SDN Controller.** The flow is now handled entirely by the distributed data plane, ensuring high performance and low latency.



The Northbound API: The Gateway to Automation

While the Southbound API (like OpenFlow) handles the low-level communication with switches, the **Northbound API** is where the true power of SDN is unleashed for developers and network operators. It's the gateway to automation and innovation.

Purpose

- To provide a **high-level, abstracted, and intent-driven API** for developers and network administrators.
- The primary goal is to **hide the complexity of the underlying southbound protocols** (like OpenFlow) and the intricacies of individual network devices. Developers specify *what* they want the network to do, not *how* to do it at a packet-by-packet level.

Common Style: REST (Representational State Transfer)

- The most common architectural style for Northbound APIs is **REST**.
- Applications interact with network resources (e.g., network topology, connected hosts, traffic flows, security policies) using standard **HTTP methods** (GET, POST, PUT, DELETE) and data formats like **JSON** (JavaScript Object Notation).
- This makes it familiar and accessible to a wide range of software developers, not just networking specialists.



Example: A Security Application

Consider a security application that needs to block traffic from a malicious IP address.

Without Northbound API: The developer would need to understand the CLI commands for every vendor's firewall/router, or craft specific OpenFlow rules for each switch.

With Northbound API: The security application doesn't need to craft complex OpenFlow rules or know device-specific commands. It can simply make a high-level API call to the SDN Controller

Controller's Role: The SDN Controller receives this high-level request, translates it into the necessary low-level OpenFlow (or other southbound) instructions, and pushes those rules down to the relevant switches in the Infrastructure Layer.

```
POST /firewall/rules
Content-Type: application/json
{
    "ip_src": "1.2.3.4",
    "action": "DROP",
    "priority": "HIGH"
}
```

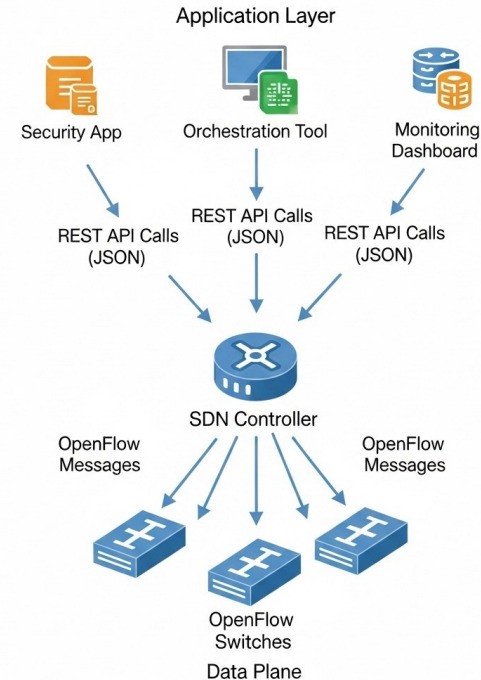
The Northbound API

This is where the true power of SDN lies.

Any developer who can make a REST API call can now programmatically control the network.

Think about the possibilities for your own projects – automating deployments, dynamically adjusting security, or optimizing performance based on application needs.

The network becomes a programmable platform, just like a server or a database.



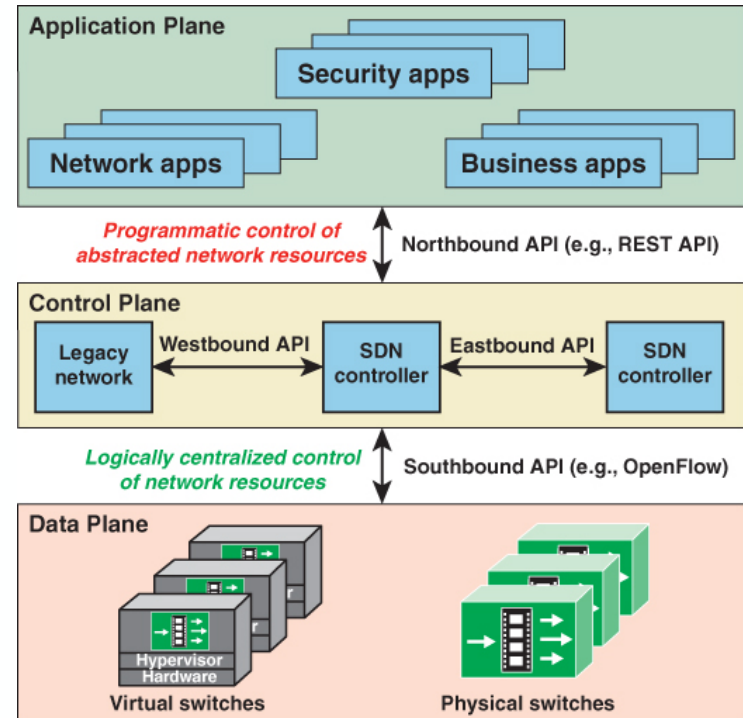
Horizontal (Peer-to-Peer) APIs in SDN

Eastbound API:

- Facilitates communication and cooperation among groups or federations of controllers.

Westbound API:

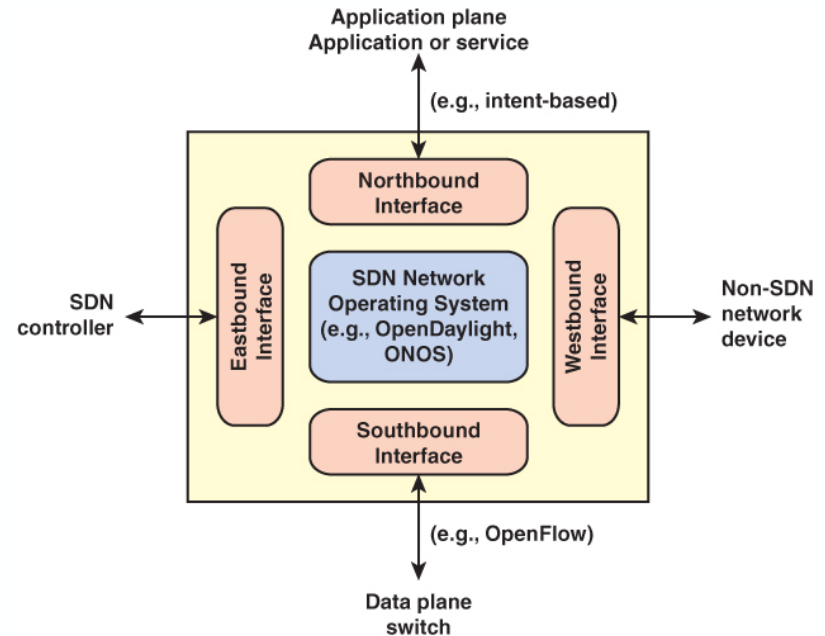
- Enables communication between SDN and non-SDN (legacy) networks.



Westbound Interface in SDN

Enables communication between an SDN controller and a non-SDN network.

Typically uses **Border Gateway Protocol (BGP)** to bridge the gap between SDN and traditional networks.



SDN Domain Structure: Eastbound Interfaces

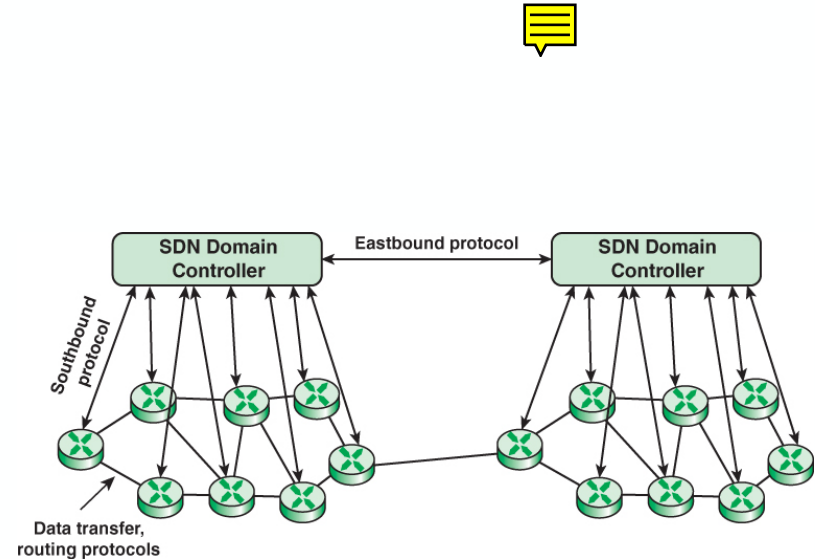
Enable information exchange among distributed controllers, supporting multiple SDN domains.

SDN Domains: Manage scalability, privacy, and incremental deployment in large networks.

Reasons for Using SDN Domains:

- **Scalability:** Multiple controllers manage large networks.
- **Privacy:** Customized policies keep sensitive info within domains.
- **Incremental Deployment:** Integrate legacy and new infrastructure flexibly.

Inter-Controller Communication: Standardized protocols for routing info exchange; no universal approach.



SDN Controller Design Patterns

The SDN Controller is the brain of the network, and its design is critical for the overall system's robustness and scalability. There are several architectural patterns for deploying SDN controllers, each with its own advantages and challenges.

1. Centralized Model

- In this simplest model, there is a **single instance of the SDN Controller** managing the entire network.
- **Advantages:**
 - **Simplicity:** Easy to deploy and manage.
 - **Consistent View:** Naturally maintains a consistent, global view of the network state.
- **Disadvantages:**
 - **Performance Limits:** Can become a bottleneck as the network size and traffic increase. A single controller might not be able to handle the volume of Packet-In messages or flow rule installations.
 - **Single Point of Failure:** If the single controller fails, the entire network's control plane goes down, leading to a complete outage.
 - **Scalability Issues:** Limited in its ability to scale to very large networks.

SDN Controller Design Patterns

2. Distributed Model (e.g., ONOS, OpenDaylight)

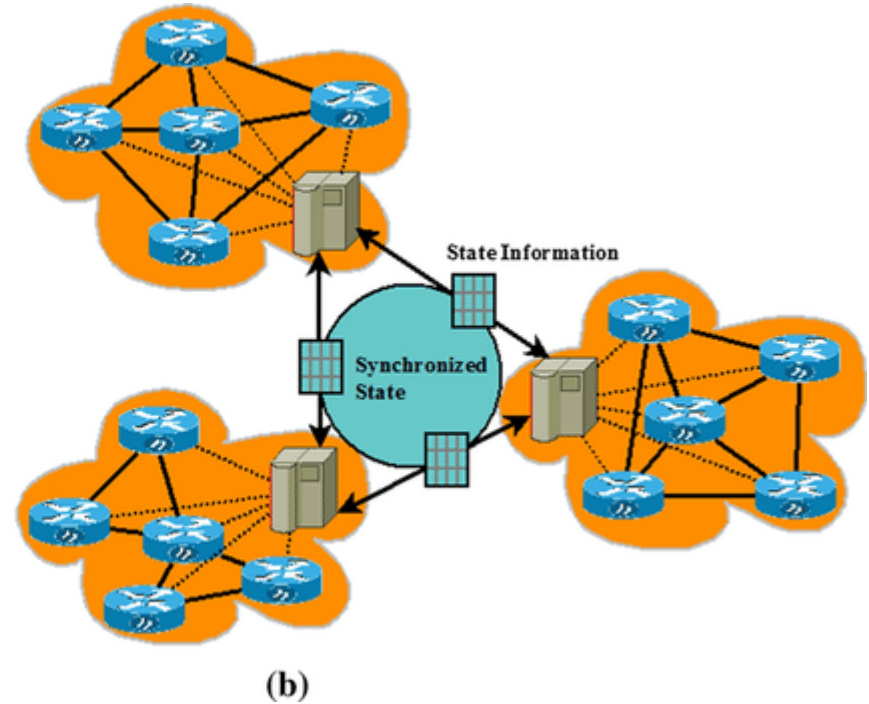
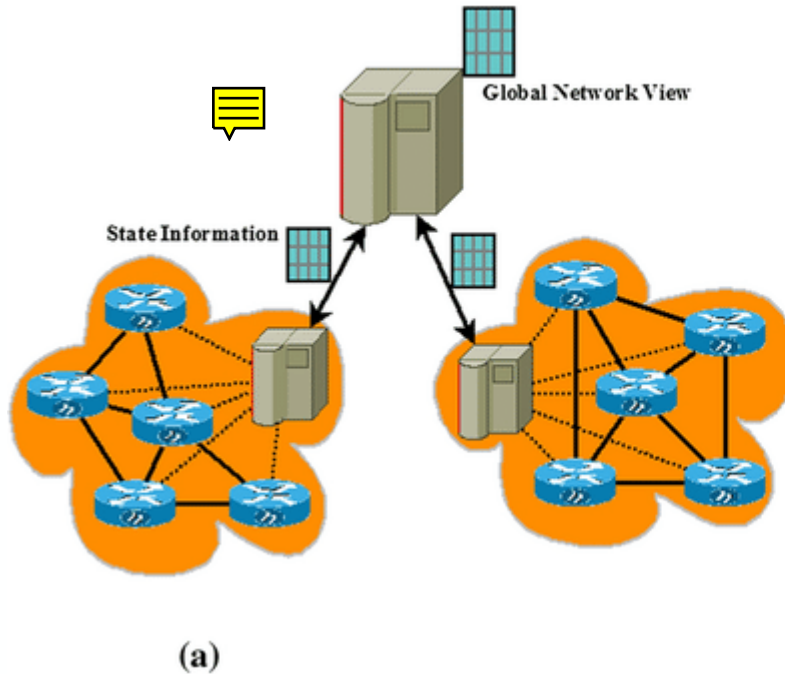
- To overcome the limitations of the centralized model, controllers are often deployed as a **cluster of controller instances** that work together collaboratively.
- **Examples:** Popular open-source controllers like **ONOS (Open Network Operating System)** and **OpenDaylight** are designed with distributed architectures.
- **Advantages:**
 - **High Availability/Reliability:** If one instance fails, others can take over, ensuring continuous operation.
 - **Scalability:** Workload can be distributed across multiple instances, allowing the control plane to scale to larger networks and higher traffic volumes.
 - **Improved Performance:** By distributing processing.
- **Key Challenge: State Management:**
 - The most significant challenge in distributed controller design is how all instances maintain a **strongly consistent, up-to-date view of the network state** (topology, device status, flow rules).
 - This is a classic and complex problem in distributed systems, often addressed using sophisticated consensus protocols like **Paxos or Raft**. These protocols ensure that all controller instances agree on the network's state, even in the event of failures.

SDN Controller Design Patterns

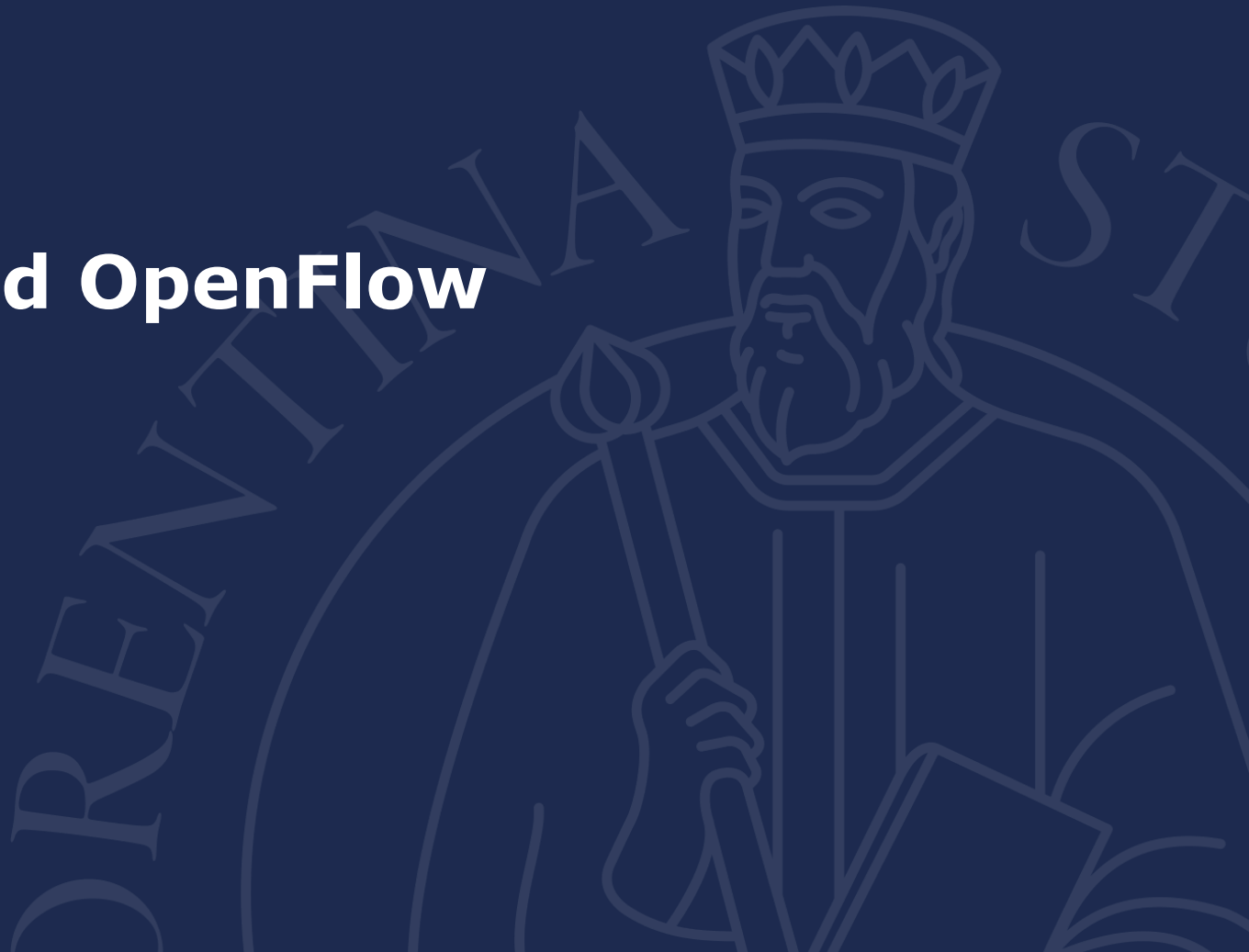
3. Hierarchical Model

- For extremely large or geographically dispersed networks, a hierarchical approach can be used, involving a "controller of controllers."
- **Architecture:** A top-level (super) controller manages multiple lower-level (domain) controllers, each of which manages a specific segment or domain of the network.
- **Advantages:**
 - **Scalability for Very Large Networks:** Breaks down a massive network into more manageable domains.
 - **Domain-Specific Optimization:** Allows for localized control and optimization within each domain.
 - **Reduced Latency:** Local controllers can respond faster to events within their domain.
- **Disadvantages:**
 - **Increased Complexity:** More complex to design, deploy, and manage due to multiple layers of control.
 - **Inter-Controller Communication:** Requires robust communication and coordination mechanisms between different controller levels.

SDN Controller Design Patterns



Beyond OpenFlow



Beyond OpenFlow: The Rise of P4

While OpenFlow provided programmatic control over *how* packets are forwarded, it still relied on switches with a fixed **set of capabilities**. To truly unlock the potential of software-defined networks, especially for 6G's demands, we need to program the forwarding hardware itself.

The Rigidity of Fixed-Function ASICs

- **Traditional Switches:** Network switches and routers have historically been built with **fixed-function ASICs (Application-Specific Integrated Circuits)**.
- **Limitations:** These ASICs are hardwired to process only a specific, predefined set of protocols (e.g., Ethernet, IPv4/6, TCP/UDP). If a new protocol emerges or a **custom header is needed**, the hardware cannot understand or process it without a costly and time-consuming hardware refresh. This stifles innovation.

The P4 Vision: Programmable Data Planes

- **What if the data plane itself was programmable?** This is the core vision behind **P4 (Programming Protocol-independent Packet Processors)**.
- **Definition:** P4 is an open-source, high-level **domain-specific language** designed to describe how a network forwarding element (like a switch or a NIC) should **process packets**. It allows developers to define custom packet parsing, matching, and action logic.

The P4 Abstract Model: **Parser** -> Match-Action Pipeline -> Deparser

P4 defines an abstract forwarding model that consists of three main stages:

Parser: You define how the switch should **recognize and extract header fields** from an incoming packet.

- **Innovation:** This includes the ability to define and parse **custom headers** or new protocol encapsulations that the hardware was not originally designed for.

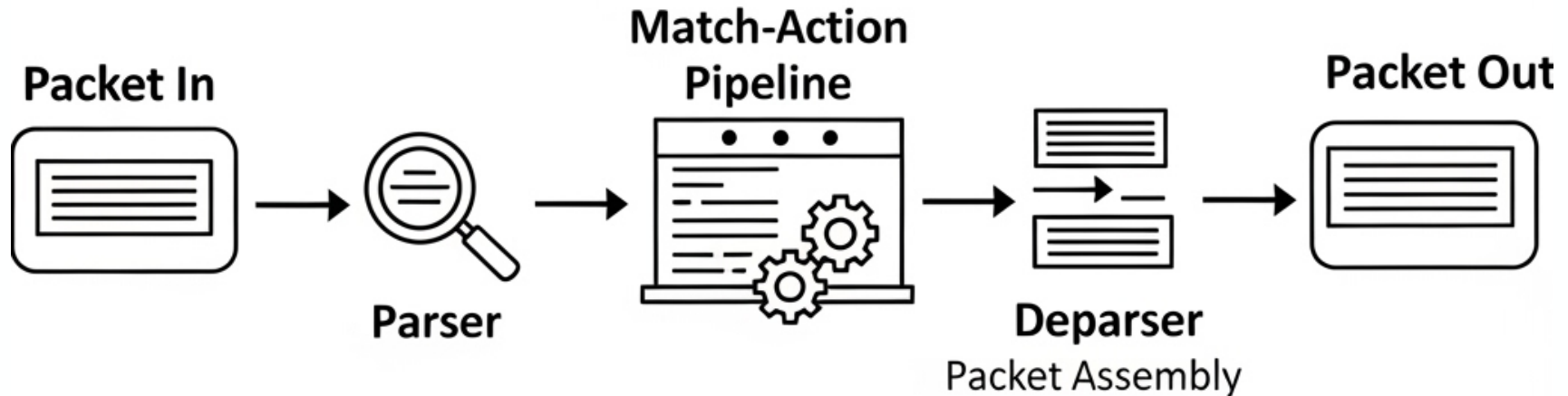
Match-Action Pipeline: This is the core processing logic. You define a series of **tables**, each with specific **match keys** (which header fields to look at) and associated **actions** (what to do if a match occurs).

- **Flexibility:** Actions can be simple (forward, drop) or complex (modify headers, update counters, generate new packets).

Deparser: You define how to **reconstruct the packet** (including any modified or new headers) before sending it out to the output port.



Packet Processing Pipeline



Impact of P4

Rapid Protocol Innovation: Enables the deployment of new network protocols and features in software, without requiring new hardware. This accelerates the pace of innovation significantly.



In-band Network Telemetry (INT): Allows network devices to embed telemetry data (e.g., latency, queue depth, path information) directly into the packet header as it traverses the network, providing unprecedented real-time visibility.



Hardware Offloading of Custom Network Functions: Enables complex network functions (like load balancing, security policies, or even parts of a firewall) to be implemented directly in the high-performance data plane hardware, significantly boosting performance and reducing latency compared to software implementations.

Mini Case Study: A Simple SDN Load Balancer



Mini Case Study: A Simple SDN Load Balancer

Let's illustrate the power of SDN by designing a simple load balancer application.

Goal: Distribute Traffic to Backend Servers

The objective is to **distribute incoming network traffic** that is destined for a single **Virtual IP (VIP)** address across multiple **real backend servers** (e.g., web servers, application servers) that host the actual service.

This ensures that no single server is overloaded and improves overall service availability and performance.

SDN Application Logic: Step-by-Step

1. Application Initialization & Configuration
2. First Packet for the VIP Arrives (Packet-In Event)
3. Controller Deliberation & Backend Selection
4. Flow Rule Installation (Flow-Mod Messages)
5. Subsequent Packet Forwarding (Line-Rate)

Mini Case Study: A Simple SDN Load Balancer

1. Application Initialization & Configuration:

- The *SDN Load Balancer application*, running on top of the SDN Controller, first needs to know:
 - The **Virtual IP (VIP)** address it needs to load balance.
 - The **IP addresses of all the backend real servers** available in the server pool.
- This information is typically configured by an administrator or another orchestration system via the **Northbound API** of the SDN Controller.

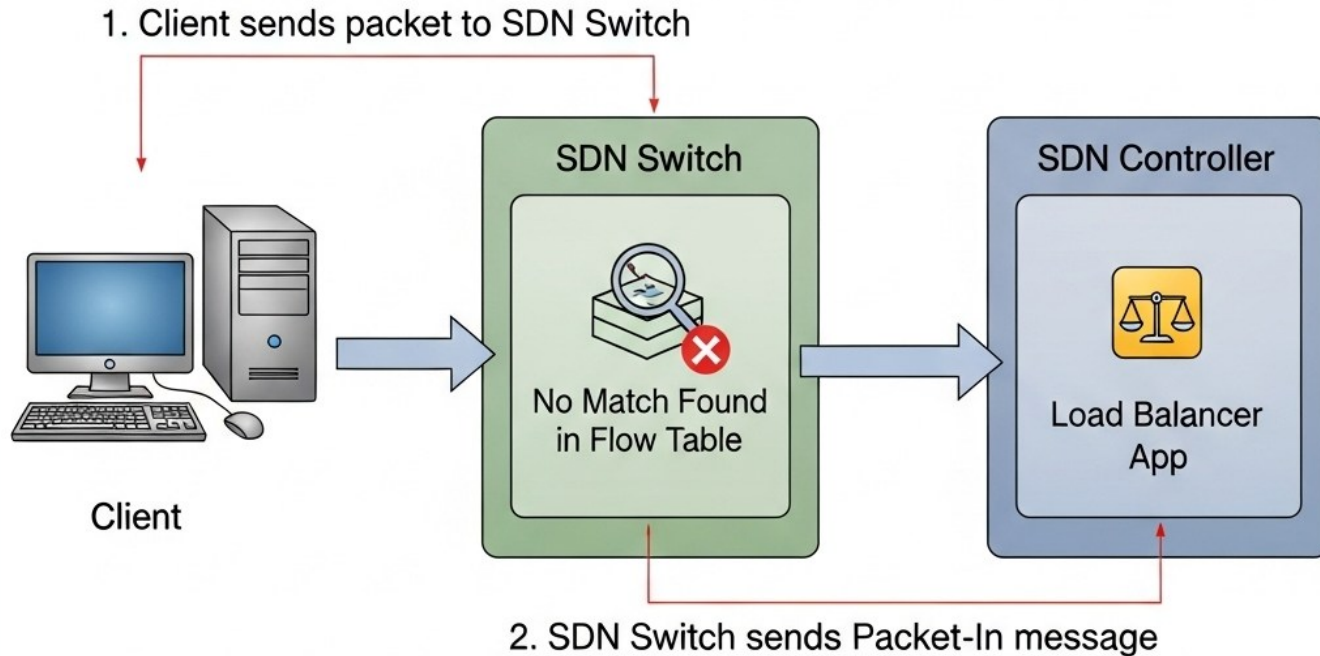
2. First Packet for the VIP Arrives (Packet-In Event):

- An end-user sends a packet destined for the **VIP**.
- Since this is the first packet of a new connection to the VIP, the OpenFlow switch at the edge of the network (where the VIP traffic first hits) will likely **not have a matching flow rule**.
- As a result, the switch sends a **Packet-In message** containing the packet's header (including the VIP as destination IP) to the SDN Controller.

3. Controller Deliberation & Backend Selection:

- The SDN Controller forwards the Packet-In event to our **SDN Load Balancer application**.
- The application analyzes the packet's header.
- It then applies its load balancing algorithm (e.g., **round-robin, least connections, hash-based**) to **choose one of the available backend real servers** to handle this new connection.

First Packet for the VIP Arrives (Packet-In Event)

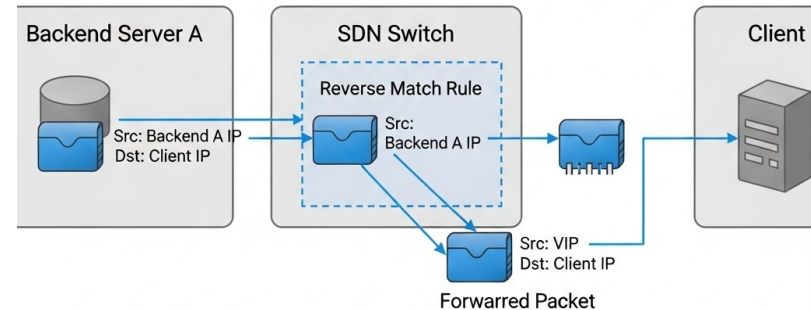
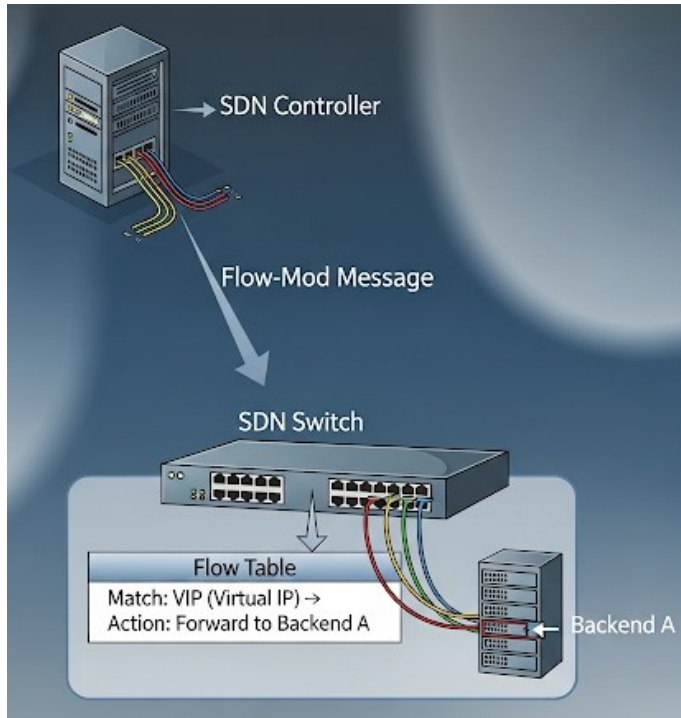


Mini Case Study: A Simple SDN Load Balancer

4. Flow Rule Installation (Flow-Mod Messages)

- Once a backend server is chosen, the SDN Load Balancer application instructs the Controller to install flow rules on the relevant switches. It typically installs **two key types of flow rules**:
 - **Forward Rule (Client to Server)**: A rule is installed on the ingress switch (and potentially other switches along the path) that matches packets destined for the **VIP** and instructs the switch to:
 - **Rewrite the destination IP address** from the VIP to the chosen **real backend server's IP address**.
 - Forward the packet to the port connected to that real server.
 - **Reverse Rule (Server to Client)**: A corresponding rule is often installed (on the same or other relevant switches) to handle the return traffic from the backend server. This rule matches packets with the **real server's IP as the source** and instructs the switch to:
 - **Rewrite the source IP address** from the real server's IP back to the **VIP**.
 - Forward the packet back towards the client.
- These rules are pushed down to the switches using **OpenFlow Flow-Mod messages**.

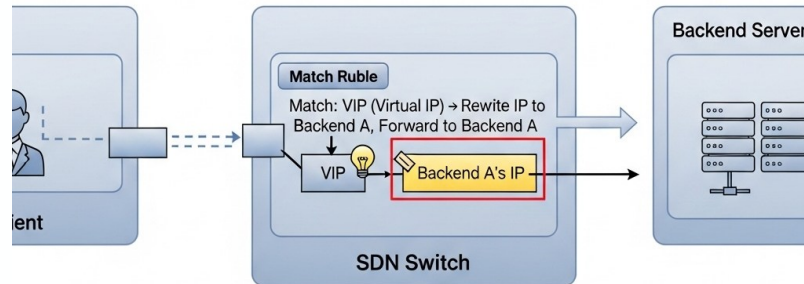
Flow Rule Installation (Flow-Mod Messages)



Mini Case Study: A Simple SDN Load Balancer

5. Subsequent Packet Forwarding (Line-Rate):

- Crucially, once these **flow rules are installed**, all **subsequent packets of the same connection (flow)** will arrive at the ingress switch and find an **immediate match** in the flow table.
- These packets are then forwarded directly by the switch's data plane at **line-rate**, without requiring any further interaction with the SDN Controller. The load balancing decision has been "programmed" into the network's forwarding hardware.



SDN Challenges

While Software-Defined Networking (SDN) has brought revolutionary changes to network management and operation, it is not without its challenges. Understanding these helps in appreciating the ongoing evolution of networking.

Scalability:

- While distributed controller designs address some scalability issues, managing extremely large-scale networks with billions of flows and dynamic changes still presents significant engineering challenges for the control plane.
- The volume of Packet-In messages from a massive network can overwhelm even a distributed controller.

Security:

- Centralizing control creates a single point of attack (the controller) and introduces new vulnerabilities in the northbound and southbound interfaces. Robust security measures are paramount and complex to implement comprehensively.

SDN Challenges

Consistency:

- In distributed controller deployments, maintaining strong consistency of the network state across all controller instances, especially during failures or network changes, is a complex distributed systems problem. Inconsistent views can lead to incorrect forwarding decisions.

Interoperability:

- While OpenFlow aimed for standardization, real-world deployments often involve a mix of OpenFlow-enabled and traditional "hybrid" switches. Ensuring seamless interoperability and managing diverse vendor implementations remains a practical challenge.
- The maturity of Northbound APIs across different controller platforms can also vary.

Summary: The Transformative Impact of SDN

Despite these challenges, SDN has fundamentally transformed the networking landscape:

- **Treating Network Control as a Software Problem:** This is the core paradigm shift. Instead of being confined to proprietary hardware, network intelligence is now in software, allowing for unprecedented flexibility and innovation.
- **Foundational Building Block:** SDN is not just a technology; it is the **foundational building block** for:
 - **Automation:** Enabling programmatic control and orchestration of network resources.
 - **Virtualization:** Providing the control layer for network virtualization concepts.
 - **The Modern Data Center:** It has been widely adopted in hyperscale data centers and cloud environments to manage their vast and dynamic infrastructures efficiently.
- **Enabler for Future Networks:** SDN's principles of decoupling and programmability are essential for realizing the vision of 5G, and even more so for the extreme demands and intelligence required by **6G networks**. It lays the groundwork for a network that can truly adapt, learn, and self-manage





UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

01: The 6G Vision

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

daniele.tarchi@unifi.it